AEROVA

INTELLIGENT SMART CITY PLATFORM

A Desktop-Style Control Center for Simulated Urban Services

Project Report

Submitted by

G. THILAK MASTHAN
REG. NO.: RA2511026020162

and

K. BALAJI
REG. NO.: RA2511026020193

Under the guidance of

_____

(Faculty, Department of Computer Science and Engineering)

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Of

COLLEGE OF ENGINEERING AND TECHNOLOGY



SRM
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University under section 3 of UGC Act, 1956)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI

BONAFIDE CERTIFICATE

Certified that the project titled “AEROVA: Intelligent Smart City Platform” is the project work of G. THILAK MASTHAN (RA2511026020162) and K. BALAJI (RA2511026020193), submitted for academic evaluation in the Department of Computer Science and Engineering for the Academic Year 2026–27.

This report documents the design, interface modules, simulated workflows, implementation approach, and proposed validation plan of AEROVA. The current deliverable is a front-end prototype. City data, sensor readings, predictions, and assistant responses are simulated in JavaScript; the report does not claim live integration with city infrastructure.

SIGNATURE

_____

Faculty Guide

Department of Computer Science and Engineering
SRM Institute of Science and Technology, Ramapuram, Chennai

Note: Faculty name, signature, and any department-required certification details should be completed by the institution before submission.

ABSTRACT

AEROVA is a front-end prototype of an intelligent smart-city control center presented through a desktop-style web interface. It brings together simulated urban-service dashboards for traffic, energy, water, waste, transport, and safety, alongside a digital city map, analytics, notifications, settings, and an AI-style assistant. The interface is designed to make multiple city-service indicators visible within one application and to demonstrate how a unified control center could support situational awareness.

The application is built with HTML, CSS, and JavaScript and is intended to run in a modern web browser without external libraries. It includes a login screen, collapsible sidebar, draggable windows, toast notifications, simulated data updates, dashboard cards, and a text-report export function. The project is a prototype rather than a production smart-city system: sensor values, traffic data, AI predictions, and map elements are simulated locally.

This report describes the problem statement, objectives, scope, architecture, modules, workflows, implementation methodology, proposed tests, limitations, and future enhancements. A production version would require a backend, database, real authentication, map/GIS services, IoT integrations, and security controls.

Keywords: Smart city, urban services, dashboard, traffic management, energy monitoring, water management, simulation, web application.

TABLE OF CONTENTS

Section	Page
1. Introduction	4
2. Scope and Requirements	5
3. System Architecture	6
4. Modules and Features	7
5. User Interface and Workflow	9
6. Implementation Methodology	10
7. Testing Plan	11
8. Evaluation and Expected Outcomes	12
9. Limitations and Future Enhancements	13
10. Conclusion and References	14

1. INTRODUCTION

1.1 Background

Cities coordinate interconnected services such as transport, traffic, energy, water, waste collection, and public safety. When service information is spread across separate tools, operators may find it difficult to build a unified view of current conditions. AEROVA demonstrates a single interface where simulated service indicators and tools can be viewed together.

1.2 Problem Statement

A fragmented monitoring experience can make it difficult to compare service status and understand how urban systems relate to one another. The project addresses this presentation problem by organizing simulated city-service information in a consistent dashboard and module-based interface.

1.3 Aim

To design and demonstrate a browser-based smart-city control center that organizes simulated urban-service information through a desktop-style interface.

1.4 Objectives

• Provide a central dashboard for key city indicators.
• Present traffic, energy, water, waste, transport, and safety modules.
• Include a simulated digital city map and analytics view.
• Demonstrate an AI-style assistant with predefined responses and prediction displays.
• Support notifications, settings, draggable windows, and text report export.
• Document limitations and a pathway toward real integrations.

2. SCOPE AND REQUIREMENTS

2.1 Included in the Prototype

The current project includes a login screen, desktop-style control center, collapsible sidebar, home dashboard, simulated city map, service modules, AI Center, analytics dashboard, notifications, settings, draggable windows, toast messages, and a text report export function.

2.2 Outside the Current Scope

The current prototype does not connect to real traffic signals, live IoT sensors, production GIS services, real emergency dispatch, or a backend database. It does not provide verified machine-learning predictions or production-grade identity management.

2.3 Functional Requirements

FR-01: Display a login screen and open the control center on successful demo submission.
FR-02: Navigate between city service modules.
FR-03: Show simulated indicators for traffic, energy, water, waste, transport, and safety.
FR-04: Present a simulated map and analytics views.
FR-05: Display notifications and toast feedback for supported interactions.
FR-06: Allow windows to be moved and the sidebar to be collapsed.
FR-07: Export a text summary from the application.

2.4 Non-Functional Requirements

The interface should be responsive, readable, easy to navigate, and usable in a modern browser. As a front-end-only application, its performance depends on the browser and device. Reliability, accessibility, security, and compatibility should be verified through systematic testing.

3. SYSTEM ARCHITECTURE

3.1 Architecture Overview

AEROVA follows a client-side web architecture. HTML defines the page structure, CSS controls layout and appearance, and JavaScript handles interactions, simulated data, dashboard updates, window behavior, notifications, and report export. The browser executes the application locally.

3.2 Main Components

Presentation layer: login screen, top bar, sidebar, dashboard cards, module windows, charts, map panel, and settings.
Interaction layer: event handlers for navigation, forms, draggable windows, notifications, and user controls.
Simulation layer: JavaScript logic that supplies sample values and demonstrates changes to traffic, energy, water, waste, and other indicators.
Export layer: creates a text report from available application information.

3.3 Data Flow

A user opens the page in a browser and submits the demo login form. The interface then displays the control center. User actions trigger JavaScript event handlers, which update visible panels or simulated values. The interface renders the result in the relevant module. In the current version, these data flows remain within the front end and do not represent live city telemetry.

3.4 Technology Stack

HTML5 is used for structure, CSS for responsive styling and desktop-like layout, and vanilla JavaScript for state and interaction logic. The project README states that no external libraries are required and that the page can be opened directly or through a local development server.

4. MODULES AND FEATURES

4.1 City Home Dashboard

Provides an overview of selected city indicators such as traffic flow, energy health, water reserves, and air quality. These are simulated dashboard values and should be interpreted as demonstration data.

4.2 Digital City Map

Presents an interactive simulated map with markers or status elements for traffic, energy, water, transport, and safety. It illustrates how geographically organized service information could be presented. It is not a live GIS map.

4.3 Traffic Management

Displays road congestion information and simulated traffic predictions. A production implementation would require validated traffic feeds, road-network data, and a defined prediction service.

4.4 Energy Management

Displays energy demand, solar generation, and grid-health indicators. Real deployment would require trusted meter or grid data and secure ingestion interfaces.

4.5 Water Management

Shows reservoir levels and simulated leak-detection information. Actual leak detection would need sensor data, threshold definitions, and alert validation.

4.6 Waste Management

Displays waste-bin capacity and demonstrates collection-route optimization concepts. Real route planning would require vehicle locations, collection schedules, road constraints, and operational data.

4.7 Transport Management

Shows bus availability, passenger loads, and fleet statistics using simulated values. Live operation would require fleet and passenger information from authorized sources.

4.8 Safety Center

Presents simulated incidents, safety scores, and emergency alerts. It is an illustrative dashboard and does not contact emergency services.

4.9 AI Center

Includes a chat-style assistant and prediction displays based on predefined or simulated logic. It should not be described as a trained AI model unless a real model and evaluation evidence are added.

4.10 Analytics, Notifications and Settings

Analytics presents visual summaries; notifications and toast messages provide interface feedback; settings provide customization options; and report export produces a text summary of available information.

5. USER INTERFACE AND WORKFLOW

5.1 Typical User Workflow

1. Open the project in a browser.
2. Enter the demo credentials shown on the login screen or submit the form as configured.
3. View the city home dashboard.
4. Use the sidebar to open a service module.
5. Inspect simulated indicators, map markers, analytics, or assistant responses.
6. Use notifications and settings where available.
7. Export a text report if required.

5.2 Interface Design

The interface uses a desktop-style control center, including a top bar, collapsible sidebar, module windows, cards, and responsive layout. Draggable windows and toast notifications are intended to make the prototype feel like a unified operations workspace.

5.3 Demo Access Note

The project README lists the demo login as admin@aerova.city and password aerova123. These are demonstration credentials only and must not be reused for any real service or exposed in a production deployment.

6. IMPLEMENTATION METHODOLOGY

6.1 Development Approach

The prototype is implemented using static web assets. The HTML file defines screens and components; the stylesheet defines visual layout and responsive behavior; JavaScript connects interactions and supplies simulated data. The README recommends opening the project in a browser or using a local server such as the Live Server extension in Visual Studio Code.

6.2 Suggested Development Phases

Phase 1: define modules and interface requirements.
Phase 2: implement the page structure and visual styling.
Phase 3: add navigation, windows, and event handling.
Phase 4: create simulated city indicators and assistant responses.
Phase 5: add analytics, notifications, settings, and report export.
Phase 6: perform browser, interaction, and responsive-layout testing.
Phase 7: plan backend and real-data integrations as future work.

6.3 File Organization

index.html contains the interface structure; style.css contains presentation rules; script.js contains client-side interactions and simulation logic; README documents features, launch instructions, and the prototype limitations.

7. TESTING PLAN

Testing Status

The following are proposed test cases, not a claim that every test has been executed. Record actual results, screenshots, browser version, and defects before marking a test as passed.

Test Cases

T01: Open the application and confirm that the login screen renders.
T02: Submit the demo form and verify that the control center appears.
T03: Navigate to every sidebar module and verify content visibility.
T04: Confirm dashboard indicators render without JavaScript errors.
T05: Verify simulated map elements and service markers are visible.
T06: Check assistant interactions and prediction displays.
T07: Confirm notification and toast interactions work.
T08: Test draggable windows and sidebar collapse behavior.
T09: Export a text report and inspect its contents.
T10: Resize the browser and check mobile/tablet responsiveness.

7.1 Evidence to Record

For each test, record the browser and device, steps, expected result, actual result, pass/fail status, and any screenshot or console error. Test results should be based on observed behavior, not assumptions.

8. EVALUATION AND EXPECTED OUTCOMES

8.1 Evaluation Criteria

Navigation completeness: all listed modules can be opened. UI responsiveness: layouts remain usable at target viewport sizes. Interaction reliability: buttons, windows, notifications, and settings behave as intended. Export correctness: generated text contains expected information. Stability: no uncaught JavaScript errors occur during common workflows.

8.2 Expected Outcomes

The prototype is expected to demonstrate a unified dashboard concept, provide a clear module structure for smart-city service views, and illustrate how simulated service indicators can be surfaced in a browser-based control center. These are project goals; measured usability or performance outcomes should be added only after evaluation.

8.3 Data Interpretation

Values shown in the prototype are simulated. They are useful for demonstrating the interface and workflow, but they should not be used for operational decisions or presented as real-time city conditions.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Limitations

- No production backend or persistent database is described in the supplied README.
- Sensor values, city data, traffic predictions, and map content are simulated.
- The AI Center uses simulated or predefined behavior rather than a verified trained model.
- Demo login credentials are not a production authentication system.
- The prototype does not control physical traffic signals, utilities, vehicles, or emergency systems.

9.2 Future Enhancements

- Add a backend API and database with access control.
- Integrate authenticated, permissioned sensor and city-service data feeds.
- Replace the simulated map with a suitable GIS provider.
- Implement secure authentication, session management, and audit logs.
- Add validated prediction models with documented datasets and evaluation metrics.
- Improve accessibility, automated tests, error handling, and deployment practices.
- Add role-based views for operators, analysts, and administrators.

10. CONCLUSION AND REFERENCES

10.1 Conclusion

AEROVA is a browser-based front-end prototype that demonstrates a desktop-style smart-city control center. Its modules bring traffic, energy, water, waste, transport, safety, analytics, notifications, and an AI-style assistant into one interface. The current project is a simulation rather than a live smart-city platform. Its value lies in demonstrating interface organization and user workflows while providing a foundation for future backend and data integration. Any production claims about accuracy, security, reliability, or real-time operation must be supported by implementation and testing.

References

[1] AEROVA project README supplied with the source files.
[2] AEROVA application source: HTML interface, CSS stylesheet, and JavaScript interaction logic supplied with the project.
[3] MDN Web Docs, HTML, CSS, and JavaScript documentation: https://developer.mozilla.org/
[4] W3C, Web Content Accessibility Guidelines (WCAG): https://www.w3.org/WAI/standards-guidelines/wcag/

Submission Checklist

Complete the faculty guide name and signature, verify institutional formatting, run the proposed test cases, attach genuine screenshots, and clearly label all simulated data. Do not claim live IoT integration or verified AI performance unless these are implemented and evidenced.